

Hamro Pasa

हाम्रो पसल

Full-Stack E-Commerce Platform

Project Architecture, Design & Development Chronicles

Timeline: Day 1 to Day 19 Summary

Completion Date: June 20, 2026

Authors: दिनेश (Dinesh) & Antigravity (AI Partner)

Version: 1.0.0 (Production Release)

Table of Contents

- [1. Executive Summary & Overview](#)
- [2. Technical Stack & Architecture](#)
- [3. Daily Development Log \(Day 1 - Day 19\)](#)
- [4. Database Schema Structure](#)
- [5. Payment Gateways Sandbox Integrations](#)
- [6. Deployment & Run Instructions](#)
- [7. Project Summary & Wrap-Up](#)

1. Executive Summary & Overview

Hamro Pasal (Nepali for "Our Shop") is a high-performance, full-stack e-commerce application designed to provide a secure, fast, and feature-rich shopping experience. The project was built from scratch, transitioning from a basic monorepo setup to a multi-container Dockerized production environment.

Key features include a secure double-token (Access + Refresh) authentication system, dynamic filtering on the product catalog, a persistent shopping cart and wishlist, coupon promo codes, authenticated user ratings/reviews, dynamic collaborative-category recommendations, administrative dashboards, local invoice PDF rendering, and multiple payment gateway integrations (Stripe, eSewa, and IME Pay).

2. Technical Stack & Architecture

The application is architected as a monorepo consisting of:

- **Backend:** Built using **NestJS** (TypeScript), structured into clean, modular features. It relies on Passport and JWT for authentication, Helmet for security headers, rate-limiting guards, and Prisma ORM to interact with the database.
- **Frontend:** React single-page application (SPA) bundled with **Vite**. State management is handled with **Zustand** (with localStorage persistence for the cart/wishlist), styled with modular vanilla CSS (supporting Dark/Light mode theme configurations), and lazy-loaded routing.
- **Database:** **SQLite** connected via **Prisma ORM**. During database transactions, atomic locks are used to preserve consistency.
- **DevOps & Infrastructure:** Dockerized with two isolated services (Backend NestJS server and Frontend static Nginx container) orchestrated through **Docker**

3. Daily Development Log (Day 1 - Day 19)

Day 1 — Setup and Architecture

- Configured monorepo scaffolding with `backend/` and `frontend/` directories.
- Initialized SQLite database with Prisma ORM and configured mock `PrismaService` for zero-setup local testing.
- Verified React-Vite local environment and smoke-tested build compilation.

Day 2 — Authentication & User Management (Backend)

- Implemented Access Token (15 min) and Refresh Token (7 days) authentication flow.
- Stored JWTs in secure HttpOnly cookies, mitigating XSS and CSRF.
- Created `@Roles(...)` decorators and guards for Role-Based Access Control (RBAC).

Day 3 — Authentication & User Management (Frontend)

- Created Zustand authentication store (`store/auth.ts`) synchronizing with cookie sessions.
- Developed `ProtectedRoute` and `AdminRoute` wrappers in React Router DOM.
- Designed a responsive glassmorphic Dark/Light mode theme switcher.

Day 4 — Product Catalog API

- Developed NestJS endpoints for product catalog management (`GET /products` public, mutation endpoints admin-only).

- Integrated category filtering, search, and pagination.
- Configured database seed with 8 initial premium items.

Day 5 — Product Catalog UI

- Created shop search, filtering sidebar, and paginated product lists.
- Designed product details view page (`/products/:id`) with layout grid and dynamic indicators.

Day 6 — Cart & Wishlist

- Created persistent Zustand cart store with automatic stock limit clamping.
- Designed slide-out Cart Drawer and overlay Wishlist Modal accessible globally.

Day 7 — Orders API

- Created transactional order endpoint (`POST /orders`) utilizing Prisma `\$transaction`.
- Enforced backend price calculation and snapshotting to prevent client tampering.

Day 8 — Checkout Flow

- Added shipping fields validation to the `Order` schema and migrated the database.
- Designed multi-step checkout form wizard and custom order history dashboards on user profiles.

Day 9 — Stripe Payments

- Integrated Stripe SDK (backend) and Stripe Elements Card inputs (frontend).
- Created Double Confirmation payment loop to transition order statuses securely to

`PROCESSING`.

Day 10 — Admin Dashboard

- Created admin statistics API aggregating sales metrics, inventories, and category distributions.
- Designed Admin tab panels for viewing metrics, transitioning order statuses, and managing products (CRUD).

Day 11 — Code Documentation

- Documented backend and frontend repositories with comprehensive JSDoc class and method details.

Day 12 — Ratings & Reviews System

- Added `Review` model to SQLite schema.
- Implemented purchase validation logic, restricting write permissions to verified purchasers only.
- Created dynamic star review selectors and aggregation counters.

Day 13 & 14 — Promo Coupon System

- Defined `Coupon` model, implementing backend validation routines (date checks, minimum spend limits).
- Integrated coupon discounts into order transactions and built admin management UI.

Day 15 — Search Relevance, Recommendations, Security & Performance

- Developed tokenized relevance-scoring catalog search (matching title weight > descriptions).
- Built category-collaborative product recommendations panel on product details.
- Integrated Helmet headers, rate-limiting guards, and React Route lazy loading.

Day 16 — Containerization & Deployment Orchestration

- Created Backend Dockerfile and Nginx-powered Frontend Dockerfile.
- Configured `docker-compose.yml` to orchestrate service links and mount database files to volume directories.

Day 17 — PDF Invoice Generation

- Integrated client-side `jsPDF` tool, allowing users to compile and download stylized order invoice PDFs.
- Rebranded application to **Hamro Pasal**.

Day 18 — Sandbox Payment Gateways Integration (eSewa & IME Pay)

- Integrated localized Sandbox payment gateways **eSewa** and **IME Pay** with payment validation and callbacks.

Day 19 — SQLite Volume Shadowing & Startup Tuning

- Resolved SQLite database volume shadowing in Docker and implemented database

transaction exception logs.

4. Database Schema Structure

The prisma schema details models representing the core components of Hamro Pasa:

```
model User {  
  
  id          String   @id @default(uuid())  
  
  email       String   @unique  
  
  password    String  
  
  name        String?  
  
  role        String   @default("USER")  
  
  hashedRt    String?  
  
  createdAt   DateTime @default(now())  
  
  updatedAt   DateTime @updatedAt  
  
  orders      Order[]  
  
  reviews     Review[]  
  
}
```

```
model Product {  
  
    id          String      @id @default(uuid())  
  
    title       String  
  
    description String  
  
    price       Float  
  
    image       String  
  
    category    String  
  
    stock       Int  
  
    createdAt   DateTime    @default(now())  
  
    updatedAt   DateTime    @updatedAt  
  
    orderItems  OrderItem[]  
  
    reviews     Review[]  
  
}
```

```
model Order {
```


id	String	@id @default(uuid())
userId	String	
status	String	@default("PENDING") // PENDING, PROCESSING, SHIPPED, DELIVERED, CANCELLED
totalAmount	Float	
discountAmount	Float	@default(0)
couponId	String?	
paymentMethod	String	@default("STRIPE") // STRIPE, ESEWA, IME_PAY
paymentId	String?	
shippingAddress	String	
shippingCity	String	
shippingPostalCode	String	
shippingCountry	String	
createdAt	DateTime	@default(now())
updatedAt	DateTime	@updatedAt
user	User	@relation(fields: [userId], references: [id], onDelete: Cascade)

```

    coupon          Coupon?    @relation(fields: [couponId], references:
[id])

    items           OrderItem[]

}

model OrderItem {

    id              String      @id @default(uuid())

    orderId         String

    productId       String

    quantity        Int

    price           Float       // Historical price snapshot

    order           Order       @relation(fields: [orderId], references: [id], onDelete:
Cascade)

    product         Product     @relation(fields: [productId], references: [id])

}

model Review {

    id              String      @id @default(uuid())

```

```

    rating      Int

    comment     String

    userId      String

    productId   String

    createdAt   DateTime @default(now())

    updatedAt   DateTime @updatedAt

    user        User      @relation(fields: [userId], references: [id], onDelete:
Cascade)

    product     Product   @relation(fields: [productId], references: [id],
onDelete: Cascade)

}

```

```

model Coupon {

    id           String      @id @default(uuid())

    code         String      @unique

    discountType String      // PERCENTAGE, FIXED

    value        Float

    minOrderAmount Float     @default(0)
}

```

maxUses	Int?	
usedCount	Int	@default(0)
active	Boolean	@default(true)
expiresAt	DateTime?	
createdAt	DateTime	@default(now())
updatedAt	DateTime	@updatedAt
orders	Order[]	
}		

5. Payment Gateways Sandbox Integrations

Hamro Pasal provides checkout support for three gateways.

Gateway	Type	Verification Process
Stripe	Global	Stripe Elements client-side payment tokens, verified via Stripe API payment intents on the backend.
	Cards	
eSewa	Nepali Wallet	Client is redirected to the eSewa Sandbox. On success, the backend processes callbacks and validates transaction details cryptographically.

Gateway Type	Verification Process	
IME Pay	Nepali	Client is redirected to IME Pay portal, and on return, payment
	Wallet	verification queries are dispatched to confirm success status.

6. Deployment & Run Instructions

Running via Docker Compose (Recommended)

Deploy the entire environment with a single command:

```
# Start all services and run database migrations/seeding automatically  
  
docker compose up --build -d
```

Checking Log Output

```
# Check Docker Compose logs  
  
docker compose logs -f
```

Running Locally (Development Mode)

To run services independently:

```
# 1. Start SQLite database and migrations in backend/  
  
cd backend
```

```
npm install
```

```
npx prisma migrate dev
```

```
npm run start:dev
```

```
# 2. Run React development server in frontend/
```

```
cd ../frontend
```

```
npm install
```

```
npm run dev
```

7. Project Summary & Wrap-Up

The development of **Hamro Pasal** represents a complete full-stack engineering lifecycle:

- **Day 1-5:** Foundation setup, user security authentication structure, and catalog interfaces.
- **Day 6-10:** Core operations including Cart/Wishlist overlays, database orders, checkout logic, Stripe, and admin consoles.
- **Day 11-15:** Advanced features including system reviews, discount coupons, search ranking, recommendations, and rate limit protections.
- **Day 16-19:** Robust deployment prep containing containerization, persistent volume layouts, client invoice downloads, localized sandbox payment gateways, and start scripts.

All unit test suites compile and pass successfully, confirming business logic verification.

The project has been fully finalized and is ready for production scaling.